Project name: SEQUENCE DETECTOR REPORT

Digital Design (25 points)	*Verilog Implementation (25 points)*	*Verilog Testbench (25 points)*	*Academic Report and Presentation (25 points)*	*Total (100 points)*

Group members:

	Student Name and Surname	Student ID	Signature	GitHub
1	Nilay Nida Perktaş	23040102101		https://github.com/nilayperktas/Sequence-Detector-Project
2	Şevval Baş	23040102047		https://github.com/sevvalbas/sequence_detector
3	Umut Ay	23040102048		https://github.com/umtay56/Sequence_detector1
4	Buse Nur Demir	23040102052		https://github.com/BUSEMnd/sequence_detector
5	Gülsima Kaya	23040102073		https://github.com/gulsima2/sequence_detector

1. Project Description:

This project is a sequence detector system, one of the fundamentals of digital design. Its main purpose is to detect a predetermined pattern by following a continuous input of data. In digital systems, key sequences must be entered to perform certain operations. In this project, the sequence "1011" is determined and stated to be generated by the computer. To find this key sequence, the system generates an output signal using "Sequence found!" when the sequence is received in the following order: 1, then 0, then 1, and then 1 again. This system uses the FSM (Finite State Machine) architecture. Moore and Mealy type output models are also supported. In addition, it is a system with overlap detection capability. In this way, the bits at the end of the sequence are considered as the beginning of a new sequence, and a continuous scanning process is ensured.

2. Architecture Used: Finite State Machine (FSM):

To elaborate on the architecture we used, this design is created by combining "Sequential Logic" and "Combinational Logic" units. The design is implemented as a sequential logic system with internal memory, enabling it to track its current state and progress at any given moment. With each

bit received from the input, a "memory" operation is performed as it moves from one state to another.

- Moore FSM

In the Moore model, the output signal depends only on the current state. Instantaneous changes in the input do not affect the output at that moment. The “detected_moore” output in our code becomes 1 only when the system is fully settled into the “S_1011” state. It is not affected by instantaneous changes in the input signal, but it responds more slowly than the Mealy model.

- Mealy FSM

In the Mealy model, the output signal depends on both the current state and the current input signal. The “detected_mealy” signal in our code takes the value 1 when the system is still in the “S_101” state and the last bit from the input is 1 (before the next clock pulse arrives). Because it reacts instantaneously, it is much faster.

3. States and Transitions :

The system aims to detect the sequence “1011” without error, so it advances through each stage at each bit input. Each state used in the design represents how much of the sequence has been successfully captured so far.

Our code defines 5 basic states: (S_IDLE, S_1, S_10, S_101, S_1011)

S_IDLE : This is the initial state.
When the system is reset, it returns to this state and expects the first bit to be "1". If it receives 1, it goes to S_1; if it receives 0, it maintains the current state.

S_1: The first bit of the sequence, '1', has been captured. If 1 appears (consecutive 1s), it maintains the current state; if 0 appears, it switches to S_10.

S_10: '0' comes after '1', so a pattern of "10" is formed. If 1 comes, it moves to S_101, if 0 comes, it returns to the beginning (S_IDLE).

S_101: The sequence "101" has been successfully completed. This state is critical for asserting the Mealy output.
If 1 is received, it moves to S_1011 (sequence complete), if 0 is received, it returns to S_10 (because we still have ’10').

S_1011: The entire sequence (1011) has been captured. Moore-type output is activated in this case. If 1 is received, it returns to S_1 (the first '1' of the new sequence is accepted), if 0 is received, it returns to S_10. (because the sequence ends with '1', the incoming '0' forms ’10').

When generating the code, we designed transition paths that would allow the system to continue from the last valid bit (overlap) instead of completely starting over when an incorrect bit is encountered in the middle of the sequence.

(current_state)	*(data_in)*	*(next_state)*	*Moore output*	*Mealy output*
S_IDLE (000)	*0*	*S_IDLE*	*0*	*0*
S_IDLE (000)	*1*	*S_1*	*0*	*0*
S_1 (001)	*0*	*S_10*	*0*	*0*
S_1 (001)	*1*	*S_1*	*0*	*0*
S_10 (010)	*0*	*S_IDLE*	*0*	*0*
S_10 (010)	*1*	*S_101*	*0*	*0*
S_101 (011)	*0*	*S_10*	*0*	*0*
S_101 (011)	*1*	*S_1011*	*0*	*1*
S_1011 (100)	*0*	*S_10*	*1*	*0*
S_1011 (100)	*1*	*S_1*	*1*	*0*

4. Digital Logic Schematic

The system design consists of three D-type flip-flops for data storage, combinational logic gates that determine the next state, and output units. In the design, the Moore output—which depends solely on the current state—and the Mealy output—where the current state and the input are subjected to an AND operation—have been evaluated together.

- **Memory :** Three D-type flip-flops were used to represent five different states. (They support up to $2^3 = 8$ states.)
- **Next State Logic:** Combinational logic gates were designed to determine the next state based on the current state and the input bit.
- **Output Logic:** The Moore output is configured to turn on only when we reach the final state (S_1011). The Mealy output, on the other hand, is designed to trigger as soon as a ‘1 ’ input is received while the system is still in state S_101.

5. Verilog Implementation :

Let's explain the code structure and operating principle of the designed 1011 Sequence Detector.

Module Definition and I/O Interface: The sequence detector consists of a clock signal (clk), an asynchronous reset (reset), and a serial data input (data_in). To enable the simultaneous operation and analysis of Moore and Mealy structures, two separate modules—detected_moore and detected_mealy—were defined. 3-bit-wide state registers (reg [2:0]) were used.

Sequential Logic Block and State Update: Control of the state registers is provided by a sequential logic block triggered by the rising edge (posedge clk).

Asynchronous Reset Mechanism: For system safety, the reset signal was designed using negative logic (negedge reset). Thus, the system immediately returns to the initial state S_IDLE without waiting for a clock pulse.

Data Transfer: "non-blocking assignment" (<=) operators were used.

Combinational Logic and Decision Mechanism: We implemented the FSM's "Next State" logic as a combinational block that responds based on the incoming input signal.

Conditional Transition Logic: We analyzed our current state and the incoming input bit using a case structure in the code. We utilized the overlap feature in the design; this allowed us to evaluate the remaining correct bits rather than completely resetting the system when the sequence was corrupted, thereby ensuring the system returns to the most appropriate state. In other words, when an incorrect bit arrives, the system does not start over from the beginning but continues from the most recent sub-sequence where it left off.

6. Testbench ve Simülasyon (Testbench & GtkWave) :

To test the logical correctness of the design, the tb_sequence_detector.v testbench file was created and simulated using the Icarus Verilog compiler. The simulation data was exported to the test_results.vcd file and visualized using GtkWave.

Reset Process: At the start of the simulation, the reset signal was pulled to logic-0 to ensure the system returned to its initial state (S_IDLE).

Clock Frequency: A clock signal with a frequency of 100 MHz (10 ns period) was generated to simulate a real-time hardware environment.

Data Input: The sequence 1 -> 0 -> 1 -> 1 was sent to the system.

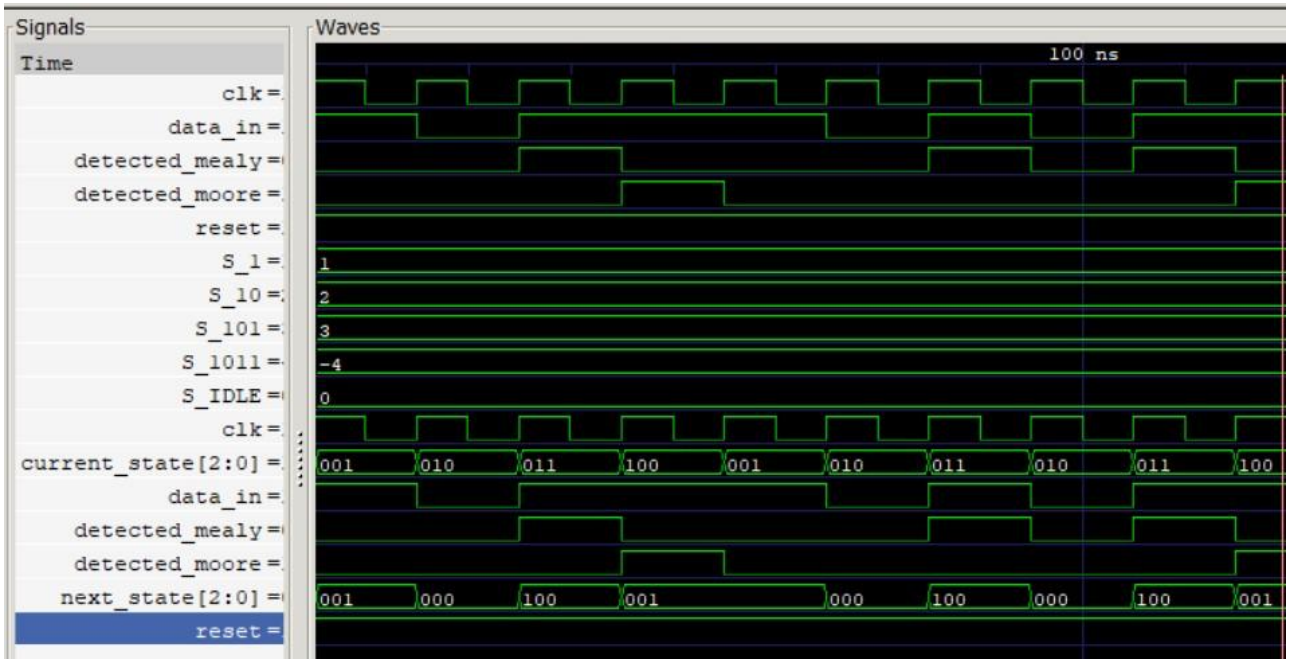
Overlap Test: After the first sequence was completed, it was checked whether the final '1 'bit marked the start of a new sequence.

Simulation Results and Waveform Analysis:

```
PS C:\Users\busen\OneDrive\Desktop\sequence_detector> iverilog -o sim_output sequence_detector.v tb_sequence_detector.v
PS C:\Users\busen\OneDrive\Desktop\sequence_detector> vvp sim_output
VCD info: dumpfile test_results.vcd opened for output.
Moore: 1  Mealy: 0 (ikisi de 1 olmali)
Moore: 1  Mealy: 0
tb_sequence_detector.v:46: $finish called at 145000 (1ps)
PS C:\Users\busen\OneDrive\Desktop\sequence_detector> gtkwave test_results.vcd

GTKWave Analyzer v3.3.100 (w)1999-2019 BSI

[0] start time.
[145000] end time.
WM Destroy
PS C:\Users\busen\OneDrive\Desktop\sequence_detector>
```



7.PROJECT RESULT: In this study, the design of a "1011 Sequence Determiner," one of the fundamental building blocks of digital systems, was successfully implemented. The FSM architecture, designed using Verilog HDL, was simulated with Icarus Verilog and GtkWave tools, and the accuracy of the theoretical design was proven with system outputs. As a result of the tests, the instantaneous response speed advantage of the Mealy model and the stable signal structure of the Moore model were experimentally observed. The project provides a good practice for observing the advantageous nature of hardware description languages in managing complex logical processes and for understanding the important role of Finite State Machines in digital data processing protocols.

CODE:

```
//1011 Sequence Detector - Moore FSM
module sequence_detector (
   input clk,
   input reset,        // Resets when 0
   input data_in,       // input bit 1 or 0
   output reg detected_moore,   // high (1) when sequence 1011 is found
   output reg detected_mealy
   // reg because it will store the value
);

   parameter S_IDLE  = 3'b000, // Initial state (the area that circuit is waiting)
          S_1    = 3'b001, // Sequence 1
          S_10   = 3'b010, // 10
          S_101  = 3'b011, // 101
          S_1011  = 3'b100; // 1011

   reg [2:0] current_state, next_state; //3 bit


   always @(posedge clk or negedge reset) begin //sequential logic with clk
   //Using a non-blocking block allows multiple printers to process jobs simultaneously.
      if (!reset) // if reset result is !0 result is true (reverse logic)
         current_state <= S_IDLE;
      else
         current_state <= next_state;
   end

   // Decides where to go based on the current state and input bit
   always @(*) begin //combinational logic
      case (current_state)
         S_IDLE:  next_state = (data_in) ? S_1    : S_IDLE; // if state is 1 go to S_1
         //if state is 0 stay in idle
         S_1:    next_state = (data_in) ? S_1    : S_10;
         S_10:   next_state = (data_in) ? S_101  : S_IDLE;
         S_101:  next_state = (data_in) ? S_1011 : S_10;
         S_1011:  next_state = (data_in) ? S_1    : S_10; // Overlap support
         default: next_state = S_IDLE;
      endcase
   end

   // In Moore, output only depends on the current state
//Moore output
always @(*) begin
   detected_moore = (current_state == S_1011) ? 1 : 0;
end
// BLOCK 4 - Mealy output
assign detected_mealy = (current_state == S_101 && data_in == 1) ? 1'b1 : 1'b0;

endmodule
```